Referee Report, Round 2: "Honest Code Benchmark" Preregistration, Specification, and Code

What I audited: the author's local working tree at `/Users/adam/dev/perf`, which is newer than GitHub. It is commit `0754d6a` (Oct. 29, 2026, 4:35 p.m. CT) plus uncommitted and untracked changes. GitHub `origin/main` has nothing newer. I copied the changes read-only as a patch (SHA-256 `778881ab…dc53`) and an archive of untracked files (SHA-256 `fcaa167e…5703`), and I rebuilt the tree on my own box. The latest author file times, in CT, are: `generate-data.js` 5:05 p.m., `preregistration.md` 5:10 p.m., `apps/server.js` 6:25 p.m., and the newest `shared-client` file 7:32 p.m. **Materials:** `preregistration.md`, `research-plan.md`, `task-specification.md` (v0.2), `generate-data.js`, `apps/` (both servers, the build script, every arm), and `harness/`. Our own review files were not audited. **Review date:** Oct. 29, 2026, from about 8 to 9 p.m. CT. This was a read-only audit. Nothing in the repository or on the author's computer was changed. "Verified" means I checked the claim by running code on the box copy, using Node 20.19.2, Python 3.13.5 and headless Chrome. **Judged against these author decisions:** (a) Arms 7, 8 and 9 share one client, taken from Arm 1, and differ only in server code. (b) Arms 6, 10 and 11 read from SQLite with `ORDER BY` and `LIMIT`/`OFFSET`. (c) The sort rule is ASCII case-insensitive, with ties broken by ascending `id` and booleans sorted true-first when ascending.

Overall assessment

The author has made real progress. The three hydration arms now ship byte-identical client bundles. The time-zone bug in the data generator is fixed. The two ID bugs that blocked edit and delete in Arms 10 and 11 are fixed. The statistical plan now uses Holm correction, a smallest effect of interest, TOST and quantile regression. The variance-triggered extra runs are gone. htmx is self-hosted, and the harness now opens a fresh context for each run, does a warmup run and records an interaction-to-settled time.

The package is still not ready to freeze, for four reasons:

1. **Arms 7 and 9 do not hydrate cleanly.** In the form, dashboard and browser archetypes, their server HTML differs from the shared client's first render. React reports hydration errors and re-renders the whole app on the client. That breaks H10's premise that Arms 7, 8 and 9 differ only on the server.
2. **The ablation still does not contain its patterns.** Arm 9 has none of the five conventional patterns. Arm 7's server tree uses 23 hook calls. Arm 8's new server tree is the cleanest of the three, which turns the intended order upside down.
3. **The SQLite decision is not implemented.** No arm uses SQLite. Arms 6, 10 and 11 still sort in memory, and all three break the canonical sort rule.
4. **The arms contaminate one another.** All arms on the Node server share one in-memory data cache. Arm 10 edits rows in place, so an edit made in Arm 10 shows up in the server HTML of Arms 6 through 9 and survives reloads.

In addition, the harness still cannot run the study. The full-run mode is unimplemented, and every scripted interaction failed on every arm I tried. The new power statement is also wrong by about a factor of two.

Recommendation: major revision. Do not freeze yet.

Status counts across the 45 tracked items (8 major and 14 minor from round 1, plus 23 from the addendum): 11 fixed, 17 partly fixed, 16 not fixed and 1 no longer applies.

Status of earlier issues

Round 1 major issues

#	Issue	Status	Evidence
M1	Ablation arms lack their patterns	Partly fixed	Arm 8's `ServerApp.jsx` and `domain.js` now build per-row classes (e.g., `Employee` with `display()` and `with()`). Arm 9's `App.jsx` still has no `createContext`, `Provider`, class, `switch`, custom `use*` hook or `ErrorBoundary`; it has about 21 hook calls and an unused `Intl.Collator` sort. Arm 7's `table/7-react-ssr-honest/App.jsx` has 23 hook calls. The clients are now identical (see A1–A4).
M2	Arms do not implement the task spec	Partly fixed	`task-specification.md` v0.2 adds Arms 10 and 11 and redefines Arm 6 to match the code. The archetype features are still missing in every arm: virtual scrolling, Chart.js, grid subtotal rows, the payment step and Luhn check, and the detail panel.
M3	Harness cannot run; selectors do not match	Partly fixed	Fixed: median and min/max, one fresh context per run (`runner.js` line 65), warmup (line 258) and new CSV columns. Not fixed: `runner.js` line 340 still says "Full run mode not yet implemented"; `metrics.js` line 3 throws; there is no randomization of run order. **Verified:** the five interaction scripts failed on all 25 combinations tested (Arms 1, 6, 7, 10 and 11 × five archetypes, at 1,000 rows, and the table again at 10,000). Examples: form uses `[name=…]` but the arms use `data-field`; grid clicks `data-row="0"` but rows start at 1; dashboard has no `startDate`; browser has no `virtual-list`; Arm 11's table has no `th[data-column]`.
M4	Measurement confounds favor SSR arms	Partly fixed	Fixed: htmx self-hosted (`server.js` line 37 onward); the timer no longer includes the data load; the 0 ms profile is capped at 10 Mbps (`profiles.js`); settled time added. Not fixed: no compression on either server; Arm 11 is still reached by a 302 redirect to port 4001 (`server.js` line 74); COOP and COEP headers are sent only by Node; the cost of the inlined `__SSR_DATA__` is still not discussed in the preregistration. The new settled metric has its own flaws (see N5).
M5	SSR arms do less work on table (ID bugs)	Fixed	**Verified over HTTP.** Arm 10 parses IDs with `parseInt` (`table/10-htmx-ejs/server.js` lines 120, 146 and 156); edit-cell returns 200 and the commit applies. Arm 11 parses in `routes.py` `_parse_state`, and `lib.py` `commit_edit` takes `row_id: int`; edit, delete (1,000 rows to 998), toggle and commit all work. The state reset part of this issue is now worse; see N2.
M6	Conflict-of-interest controls not auditable	Partly fixed	The COI statement now names the same arms throughout. There is still no independent programmer, no AI-assistance disclosure, no time log, no build history and no freeze DOI.
M7	Statistical families and null tests	Partly fixed	Fixed: Holm; SESOI (lines 210–212); TOST for H10 and H12; a test for H4 (line 244); quantile regression with run order (line 238); CV rule removed (line 123); analysis script promised before collection (line 281); run count corrected to 82,500 (line 300). Not fixed or new: the power claim (line 121) is wrong (N4); the 100-test H1 family leaves out the device and latency cells; CLS is listed both as analyzed and as "not analyzed"; H6 and H7 compare different metrics (N5).
M8	Data generation not reproducible	Partly fixed	**Verified.** `generate-data.js` lines 123–124 use `Date.UTC`. Output under UTC, America/Chicago, Asia/Tokyo and Pacific/Kiritimati is byte-identical and matches the checked-in files. Not fixed: there is no `package-lock.json` in `apps/` or `harness/`; `apps/package.json` uses caret ranges; there is no Python requirements file; and no hash manifest covers the data or vendored scripts.

Round 1 minor issues

#	Issue	Status	Evidence
1	"Every payload size" for bundle size	Partly fixed	H8 dropped the phrase; H3 (line 35) still has it.
2	Old arm name in H6 analysis	Fixed	"htmx+genX+domx arm" no longer appears.
3	Arm 6 compared with Arm 9	Partly fixed	Line 21 now says Arm 7; line 99 still says "against Arm 9."
4	H12 "same HTML fragments"	Partly fixed	Now "equivalent HTML fragments" with TOST at 20 percent. The templates still differ, and Arm 11 still loads genX and domx.

#	Issue	Status	Evidence
5	Blinding; analysis script	Partly fixed	A script is promised before collection (line 281). Blinding is still "not applicable" (line 79), and no script is in the repository.
6	TTFB claim not validated	Not fixed	Line 185 is unchanged, and there is no fixture.
7	p98 of about 10 interactions is the maximum	Partly fixed	Per-interaction settled times are now recorded. The preregistration still reports p98 without saying it equals the maximum.
8	`rows` not validated	Fixed	`ALLOWED_SIZES` in `server.js` line 66 and in `fastapi-server.py`.
9	EJS caching	Fixed	Arm 10 renders with `{cache: true}`.
10	Single worker and CPU sharing unstated	Not fixed	The preregistration does not mention uvicorn workers or CPU sharing.
11	128 MB of data in git	Not fixed	`data/` is unchanged.
12	VPS details	Not fixed	The provider, instance type, kernel, Chrome version and frequency scaling are still missing.
13	Mobile profile throttles CPU only	Not fixed	Not stated.
14	No threats-to-validity section	Not fixed	None found.

Addendum items (shared client and SQLite)

#	Item	Status	Evidence
A-1	One shared client per archetype, taken from Arm 1	Fixed	`apps/<archetype>/shared-client/App.jsx`. Compared with the original Arm 1, it changes only the sort rule and the locale (0–23 changed lines per archetype).
A-2	Identical `index.jsx` with `hydrateRoot`	Fixed	Arms 7, 8 and 9 each import `../shared-client/App.jsx` and pass `initialData={window.__SSR_DATA__}`. Arm 1 uses `createRoot`.
A-3	Equal bundle hashes for 7, 8 and 9	Fixed	**Verified.** Hashes are identical within each archetype (table `6abb5063…`, form `9b74eba2…`, dashboard `3fab5714…`, browser `e243d8f3…`, grid `0e4a9246…`).
A-4	Client-only patterns removed from 8 and 9	Fixed	Memo, local edit state, the reducer, the debounce and the fetch fallback are gone from the client, because the client is shared.
A-5	Server HTML byte-identical to the shared client's first render	Partly fixed	**Verified.** Arm 8 matches in all five archetypes at all five sizes. Arms 7 and 9 match in table and grid only. See N1.
A-6	Arm 7 server tree as pure functions with no hooks	Not fixed	`table/7-react-ssr-honest/App.jsx` has 23 hook calls. Spec line 33 applies the no-hooks rule to Arm 7.
A-7	Arm 8 per-row classes on the server; raw JSON to the client	Fixed	`ServerApp.jsx` and `domain.js`; `server.js` line 130 serializes the raw `data`.
A-8	Arm 9's five patterns in the server tree	Not fixed	None present (see M1). The spec (line 57 onward) still promises five.
A-9	Canonical sort in every arm	Partly fixed	**Verified** on the table at 100,000 rows, all columns, both directions. The shared client (`App.jsx` lines 84–95) and CSR Arms 2–5 show 0 differences. Arm 10 uses `localeCompare` with no `id` tie (`server.js` line 36); the email column differs in 258 rows, first at sorted position 4,200 ascending. Arm 11's `lib.py` line 50 sorts case-sensitively with booleans false-first, so every row of `active` differs. Arm 6 still uses `localeCompare` (`App.jsx` line 161).
A-10	`toLocaleString('en-US')`	Partly fixed	Changed only in the shared client. Arms 2–6 and Arm 10's `_content.ejs` line 88 still call `toLocaleString()` with no locale.
A-11	Integer cents for money	Not fixed	Grid `ql` (e.g., 24011.85) and dashboard `amount` (e.g., 1497.04) are still floats.
A-12	One salary parse rule	Not fixed	JavaScript `parseInt`, with a fallback to the old value, turns "12.5e3" into 12 and can never store 0. Python `int()` rejects "12abc" but accepts "1_000" and "0".

#	Item	Status	Evidence
A-13	SQLite file, schema, indexes and manifest	Not fixed	No SQLite anywhere (`rg -i sqlite` finds nothing in the code).
A-14	`instr` filter, whitelisted `ORDER BY`, `LIMIT`/`OFFSET`	Not fixed	Arms 6, 10 and 11 filter, sort and page arrays in memory.
A-15	Matching driver settings and recorded versions	Not fixed	Depends on A-13.
A-16	Per-run working copy and reset endpoint	Not fixed	Arm 10 resets with a shallow copy (`tableData = [...data]`, line 77; grid line 69); Arm 11 uses `table_data = list(data)` (`routes.py` line 73). See N2.
A-17	Parse IDs at the route boundary	Fixed	See M5. A non-numeric ID (`id=abc`) gives a 500 error in Arm 11 (minor).
A-18	Split Server-Timing (`load`, `map`, `render`, `serialize`)	Not fixed	`server.js` lines 97–100 and 114–117 send only `render;dur`. `JSON.stringify(data)` (line 130) is not timed. FastAPI times render only (lines 88–91).
A-19	CI checks for bundle hash, markup parity and hydration	Not fixed	No test or CI file performs them. Twenty-eight untracked build artifacts (`apps/arm8-*-<timestamp>.mjs`, `apps/shared-*-<timestamp>.mjs`) suggest that an ad hoc check compared only Arm 8 with the shared client.
A-20	Preregistration amendments (H10, data access, data files)	Partly fixed	H10 adds TOST on INP with a 10 ms bound (line 252). There is no data-access paragraph, and line 101 still says "Static JSON data files (identical across all arms)."
A-21	Stale spec passages (lines 21 and 61)	Partly fixed	The old line 61 is replaced by line 67 (client-side after hydration). Line 21 still says "The client-side integration differs between all four SSR arms," which contradicts line 67.
A-22	Rebuild Arm 6 to spec before SQLite	No longer applies	The spec now defines Arm 6 as React SSR with a stateless-domx client, which matches the code.
A-23	Same initial data source for 7, 8 and 9	Fixed	All three use `loadData` in `server.js` lines 24–32. That is JSON, not SQLite.

New issues

Major

N1. Arms 7 and 9 fail hydration in three archetypes (verified). I compared each arm's server HTML with `renderToString(<App initialData={data} />)` from the shared client at 100, 1,000, 10,000, 50,000 and 100,000 rows. Arm 8 matches everywhere. Arms 7 and 9 differ in form, dashboard and browser. In dashboard and browser, the text-node layout differs: the shared client emits 30 and 29 `<!-- -->` separators, while the server trees emit 3 and 4. In form, the Arm 7 and 9 servers also prefill field values (`value="Andrew"`), while the shared client renders empty fields and fills them in an effect. In headless Chrome at 1,000 rows, Arms 7 and 9 throw React errors #425 and #418 (8, 8 and 28 errors in form, dashboard and browser). React then throws away the server DOM and renders the whole app again on the client. Arm 8 shows no errors, and table and grid are clean for all three arms. As built, the client metrics for Arms 7 and 9 include a full client render that Arm 8 does not do, so H10's client equivalence test would compare different work. **Fix:** write each server tree to the shared client's text layout and initial values, and add a browser hydration check that fails on any recoverable error.

N2. The arms contaminate one another through a shared data cache (verified). `server.js` lines 24–32 cache each data file in memory and hand the same objects to every arm. Arm 10's table copies only the array (`[...data]`, line 77) and then edits row objects in place (line 161, `row[col] = …`). Grid Arm 10 does the same on commit and bulk fill (lines 132–144). After one Arm 10 commit-edit, the edited value (`ZZZ_EDITED`) appeared in the server HTML and `__SSR_DATA__` of Arms 6, 7, 8 and 9. It survived an Arm 10 reload and was still present at the end of the audit. The `col` parameter is not checked: `col=__proto__` returned 200. Any run order that puts a table or grid mutation before

another arm changes that arm's input. **Fix:** copy rows deeply on reset, or move to the per-run SQLite copy (A-16). Whitelist editable columns, and check data hashes before each run.

N3. The ablation order is now inverted. Arm 8's new server tree uses pure functions with no hooks, plus per-row classes. Arm 7, the Honest Code arm, renders a tree with 23 hook calls, and Arm 9 has none of its five patterns. H10 predicts Arm 7 < Arm 8 < Arm 9 in server time and reads the gaps as the cost of classes and of the "tree-depth patterns." As built, Arm 8 is closer to the Honest Code rules than Arm 7, so the reading of those gaps does not follow. **Fix:** give Arm 7 the hook-free tree Arm 8 now has, without the classes, and build Arm 9's five patterns into its server tree.

N4. The new power statement is wrong. Line 121 claims about 80 percent power at Cliff's delta 0.45 with 30 runs per group under Holm across 100 tests. Delta 0.45 corresponds to roughly d = 0.85. Holm's first step tests at α = 0.05/100 = 0.0005, two-sided. At n = 30 per group, the power is about 0.4. Eighty percent power at that α needs roughly 55 to 60 runs per group, or a larger effect. **Fix:** correct the figure, or raise the run count, or narrow the family.

N5. The settled metric is fragile, and H6 and H7 mix metrics. `harness/src/collectors/web-vitals.js` (line 20 onward) watches `documentElement` and takes the last mutation before the next interaction starts. There is no quiet window, so unrelated mutations count, such as a tooltip, a focus ring or a late image. The window after the last interaction never closes, and with about five interactions, p98 is the maximum. The preregistration has H6 and H7 compare Arm 10's settled time with the CSR arms' INP, which are different measures. **Fix:** end each window after a fixed quiet period or at an app-set `data-ready` marker, scope the observer to the app root, and compare the same metric across arms.

Minor

- **N6.** The preregistration (line 75) still calls the 0 ms profile uncapped, but `profiles.js` now caps it at 10 Mbps.
- **N7.** New contradictions in the spec. Line 19 says Arms 6 and 7 share server components, but Arm 6 renders its own `App` with 18 hook calls. Line 21 conflicts with line 67. A statelessness and `renderToString` paragraph sits under FastAPI (line 29). Line 190 refers to "SSR fragment boundaries (arms 6-8)" and "three SSR arms." Line 254 says Arm 6 isolates DATAOS against Arm 7, though their server code differs. Line 256 says "identifies." Lines 97–98 still promise a shared virtual-scroll utility and Chart.js.
- **N8.** `research-plan.md` line 82 has a broken sentence: "holding the rendering mode fixed, To our knowledge…"
- **N9.** The locale is now split. Arm 1 formats in `en-US`, while Arms 2–6 and 10 use the default locale, so Arm 1's output can differ from its CSR peers on a non-US machine.
- **N10.** `__SSR_DATA__` is inlined as raw `JSON.stringify` inside a `<script>` tag (`server.js` line 130). A `</script>` in any string would break the page. The data are generated, so the risk is low, but escape `<`.
- **N11.** There is dead code in the ablation arms. Arm 8's `App.jsx` is no longer imported, and Arm 9 has an unused `Intl.Collator` sort. Remove both so any line or pattern counts reflect what runs.
- **N12.** Arm 11 returns 500 for a non-numeric ID, and Arm 10's `parseFloat` and `parseInt` accept trailing junk. Validate inputs the same way in both runtimes.

Review by area

1. Experimental design and confounds

The shared client is a clear improvement. However, N1 means two of the three hydration arms still run different client work in three archetypes. N2 adds a new confound that depends on run order. The archetype features that were

supposed to create load are still missing (M2), and the run order is still not randomized (M3).

2. Statistical plan

This is much stronger: Holm, SESOI, TOST, quantile regression with run order, a fixed run count and a test for H4. Remaining problems: the power figure (N4); the H1 family leaves out the device and latency cells; CLS appears as both analyzed and "not analyzed"; and H10's equivalence test covers INP only, not TTI or settled time.

3. Falsifiability of hypotheses

H9 now has a disconfirmation rule, and H6 and H7 now use settled time. H10 cannot be read as planned until N1 and N3 are fixed. H6 and H7 should compare the same metric across arms (N5).

4. Scope and limitations

There is still no threats-to-validity section, no VPS or Chrome detail, and no statement that the mobile profile emulates CPU only. The preregistration does not yet say that the SSR arms ship the full dataset, or that data access is part of the rendering-mode variable.

5. Conflict-of-interest management

The wording is now consistent. The substantive controls are still absent: an independent programmer, an AI disclosure, an effort log and a freeze DOI.

6. Code-to-registration alignment

The status table above covers this area in detail. The largest gaps are SQLite (A-13 to A-16), the canonical sort in Arms 6, 10 and 11 (A-9), Arm 9's patterns (A-8), and the line 101 data-files sentence (A-20).

7. Fit for *Software: Practice and Experience*

The fit has not changed: a useful practice-oriented study once the arms match the spec and the harness runs end to end. A pilot log covering all 55 combinations should come before the freeze.

8. Advocacy language

The preregistration's line 21 "intended to show" wording is fixed, "minority pattern" is gone, and the grid wording in the spec is softened (lines 149 and 168–170). Spec line 256 ("identifies") still states a conclusion before the data exist.

Verification results

Check	Result
Generator determinism (UTC, America/Chicago, Asia/Tokyo, Pacific/Kiritimati)	Identical output; matches the checked-in data
Bundle identity, Arms 7–9, five archetypes	Identical SHA-256 within each archetype
Server HTML vs. shared client, five sizes	Arm 8 matches everywhere; Arms 7 and 9 match only in table and grid
Hydration in headless Chrome, 1,000 rows	Arm 8 clean; Arms 7 and 9 throw #418 and #425 in form, dashboard and browser
Sort parity, table at 100,000 rows	Shared client and Arms 2–5 match; Arm 10 email differs in 258 rows; Arm 11 `active` reversed; Arm 6 uses `localeCompare`

Check	Result
ID handling over HTTP	Arms 10 and 11 edit, delete and commit correctly
Cross-arm state	An Arm 10 edit appears in Arms 6–9 and persists
Harness interaction scripts	0 of 25 arm-archetype combinations completed
SQLite parity	Not possible: SQLite is not implemented

What I did not verify

- A full harness run, or any real INP, TTI or settled-time numbers. The runner's full mode does not exist.
- Conditions on the study VPS, including CPU, Chrome version and frequency scaling.
- Arm 11 reached through the Node server's 302 redirect in a browser. I tested it directly on its own port.
- Hydration at sizes other than 1,000 rows. The HTML comparison covered all five sizes.
- Interaction scripts on Arms 2–5, 8 and 9. They share Arm 1's markup or client, so the same selector failures are expected.
- The grid, dashboard and browser sort rules. The sort check covered the table archetype only.

On the author's computer, the read-only copy step left temporary files in `/tmp` (`hcb-review-wt.patch`, `hcb-review-untracked.tgz`, `hcb-untracked.txt` and `hcb-data-sha.txt`) and tool output under `~/agent-tools/`. Nothing was written inside `~/dev/perf`.